

Week 12 demo — fallback captures

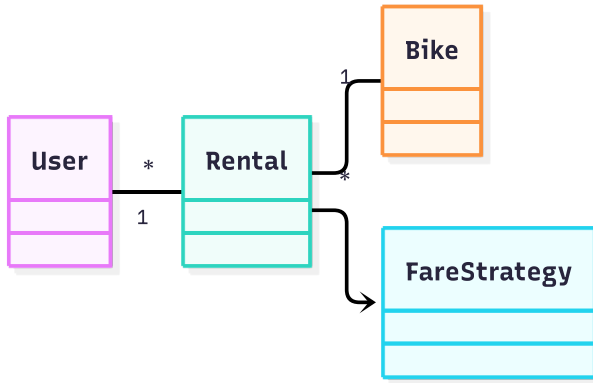
Ask AI to defend a design

Captured with Claude Code, Sonnet 5 (low effort), September
2026

When to use this deck

Switch here if the live AI fails. These are real answers to the demo's own prompts; walk the same failure catalogue against them, then say aloud what a human answer would sound like.

The design to defend



Prompt #1

“Defend this bike-sharing design as if you were the student in an oral exam: why did you make these choices, and is the design correct? [design pasted]”

Al's defense — why these choices

Short answer: the skeleton is right, but as drawn it's incomplete. Here's my defense, and where I'd concede weaknesses.

Rental as its own class (not a direct User–Bike link). A user rents a bike many times, and a bike is rented by many users over time. That's a many-to-many relationship, and it carries data of its own: start/end time, start/end station, and cost. Data that belongs to the relationship belongs in an association class. `Rental` is that class. [...]

FareStrategy as a separate abstraction (Strategy pattern). Pricing rules change often: per-minute, flat rate, subscriber discounts, peak pricing. I don't want `Rental` to contain `if (type == ...)` logic. `Rental` depends on the `FareStrategy` interface, and I can add new fares without modifying `Rental` (Open/Closed Principle).

Directed association `Rental` → `FareStrategy`. `Rental` knows its strategy. The strategy doesn't need to know about rentals, so there's no back-reference and low coupling.

Al's defense — is it correct?

Structurally yes, but I'd concede these gaps:

1. **No attributes or operations.** [...]
2. **FareStrategy isn't shown as an interface.** The Strategy pattern needs concrete subclasses (PerMinuteFare, FlatFare, SubscriberFare) with generalization arrows. [...]
3. **Missing multiplicity on the FareStrategy edge.** It should be Rental "*" --> "1" FareStrategy. Strategies are shareable, and stateless strategies can be reused across rentals.
4. **Bike multiplicity is arguably too loose.** Rental * - 1 Bike is right. But a bike can only be in one *active* rental at a time. That's an invariant (an OCL constraint or note), not something multiplicity can express.
5. **Missing domain concepts.** Station/Dock, Payment, and possibly Subscription are absent. [...]

[...]

Bottom line

The core decisions (association class for Rental, Strategy for pricing, directed dependency) are sound and justified. The diagram as drawn is a correct sketch but not a complete design. I'd fix items 1–3 immediately and be ready to discuss 5–7 as design decisions.

Prompt #2 — the examiner's follow-up (same session)

“Read the arrow between Rental and FareStrategy aloud: what exactly does it mean in UML? And which alternative to Strategy did you reject, and why?”

Follow-up — reading the arrow

Rental --> FareStrategy is a **navigable (directed) association**.

Read aloud: “A Rental holds a reference to a FareStrategy and can navigate to it; a FareStrategy does not know about Rental.”

- ▶ **Solid line with an open arrowhead** means association with navigability in one direction only. [...]
- ▶ It is **not a dependency**. A dependency is a dashed line with an open arrowhead, meaning “uses transiently”, for example as a parameter or local variable.
- ▶ It is **not aggregation or composition**. There is no diamond, so Rental does not own the strategy or control its lifetime. [...]
- ▶ **Multiplicities are unspecified**. The diagram doesn't say how many Rentals point to one strategy. I intended $* \rightarrow 1$, but as drawn it's undefined, which is a flaw I should fix.

One caveat I should own up to: in UML 2, an unmarked association end is ambiguous. [...] The arrowhead here removes that ambiguity, which is why I used it.

Follow-up — “the alternative I rejected”

Option: put fare logic inside Rental with a type flag or subclasses of Rental (MemberRental, CasualRental, each overriding `computeCost()`).

Why I rejected it:

1. **It conflates two axes of variation.** [...] That leads to class explosion (PeakMemberRental, and so on).
2. **A rental can't change strategy at runtime.** [...]
3. **A type flag with if/switch** violates Open/Closed. [...]

Trade-off I accept: Strategy adds an extra class and indirection. If there were only two fixed fares that never changed, a simple conditional would be more honest. I chose Strategy because pricing is the most volatile part of this domain.

A closer alternative I considered: a **Decorator** or rule-chain for stacking discounts (base fare plus member discount plus promo). [...]

Make-it-flip challenge (fresh session)

“Actually, is the multiplicity between Rental and Bike correct? Are you sure? A reviewer says this design is wrong. Defend it, or concede.”

I'm sure, and I'd defend it. Rental "*" -- "1" Bike reads: each rental involves exactly one bike, and a bike can appear in many rentals over its lifetime.

[...]

2. **“A rental could cover multiple bikes, such as a group or family rental.”** That's a legitimate requirement, and it would make the end 1..*. But nothing in the problem statement suggests it. If the spec did include it, I'd concede that point and change the diagram. [...]

Verdict: I'd defend it and ask the reviewer which specific scenario breaks it. If they name group rentals or mid-trip swaps as in scope, I'd revise. Otherwise the multiplicity stands.